A Prover's Share of Mathlib: Measuring Premise Coverage Without Running Lean

Anirudh Balaji
PES University
Bengaluru, India
PES1UG25AM042@stu.pes.edu

Abstract

A Lean proof the kernel accepts is correct; it is also a *selection* of premises from a library, and `pass@k` is blind to that selection. We ask what can be measured about it from released artifacts alone (a published mathlib dependency graph and pre-verified proof corpora, with no Lean invocation), and how reliably. The instrument needs characterizing first. The release's explicit-edge flag does not mean "written in source", so no source-level extractor, however good, reaches unit retention against it. We therefore report retention as a scope decomposition, at 0.6124, of which 0.8815 is what a per-declaration instrument can reach and the rest is not addressable by reading source. That efficiency is flat across tactic classes, so the instrument does not fail preferentially on the proof styles that separate the corpora. We quantify the displacement our own extractor induces, and show that proof length must be measured on a wrap-invariant axis, because character counts measure source formatting rather than proof size. Measured with that instrument, mathlib's proofs draw on 78,642 distinct premises against 1,127 for a released machine corpus: 69.8× on the full corpora, 28.5× restricted to the library regions the machine vocabulary occupies, and 18.8× matched on proof count, tactic class and proof length, with the human-only fraction falling from 98.7% to 96.3% across those designs. Only 113 premises are machine-only, and 25.6% of the machine vocabulary is language infrastructure rather than mathematics. A depth finding we built on a released per-declaration attribute is withdrawn, because the attribute is a centrality proxy. The one-line check that would have caught it is to correlate a derived attribute against in-degree and inspect its extremes.

1 Introduction

A Lean proof the kernel accepts is correct, and nothing more needs saying about that. But a proof is also a *selection*. Of the hundreds of thousands of declarations in a library, it reaches for a particular few. That selection determines what a premise-selection model must learn to retrieve, what breaks when a lemma is renamed or generalized, and whether a development participates in the library or merely sits beside it. Correctness is settled by the kernel. Nothing settles the selection.

We make it measurable. A proof's premises are nodes in the library's dependency graph, so a proof population induces a subgraph, and two populations can be compared by which parts of the library their subgraphs occupy. We compare a released machine corpus of 29,750 Lean proofs against mathlib's 70,086 declarations, working entirely from published artifacts and invoking Lean nowhere.

Three contributions follow. First, what a released-artifact pipeline can measure and how reliably (Section 3): the release's `is_explicit` flag does not mean "written in source", so no source-level instrument should be expected to reach unit retention against it; we report retention as a scope decomposition at 0.6124, of which 0.8815 is what a per-declaration instrument can reach, flat across

NeurIPS 2026 Workshop on AI for Verifiable Coding.

tactic classes so the instrument does not fail preferentially on the proof styles that separate the corpora; we quantify the displacement our own extractor induces, and show that proof length must be measured on a wrap-invariant axis. Second, the coverage result (Section 5): mathlib's proofs draw on 78,642 distinct premises against the machine corpus's 1,127, a gap that survives every control we can construct at 69.8× on the full corpora, 28.5× restricted to the machine vocabulary's own library regions, and 18.8× matched on proof count, tactic class and length. The direction is invariant across designs and the magnitude is not, so we quote the range throughout. Third, a caution (Section 6): a depth finding we built on a published attribute is withdrawn, because the attribute is a centrality proxy, and we give the check that would have caught it.

2 Background

mathlib4 and why its dependencies form a graph worth studying. Lean 4's mathematical library is a single shared development whose declarations are each checked against a kernel, so the proof term one elaborates to names exactly the other declarations it depends on. "What is used to prove what" is recorded mechanically across the whole library, with no citation practice or bibliographic convention standing between the record and the reasoning.

The published graph. Li et al. [2026] analyse that structure at three granularities (declarations, the modules that contain them and their imports, and namespaces) and release the graph itself. We consume the declaration layer and do not rebuild it: 308,129 declarations and 8,436,366 dependency edges, pinned at mathlib commit 534cf0b (February 2026). Working from the release rather than from our own extraction is what makes the human side of this paper comparable to theirs.

Most dependency edges were never written by anyone. The finding that shapes this work is that only 2,178,534 of those edges (25.8%) are *explicit*; the remaining 74.2% are synthesized during elaboration, arriving through instance resolution, unification and tactic machinery rather than from anything in the source a contributor typed. A dependency graph read whole is therefore substantially a record of what the compiler did. Li et al. identify the explicit subgraph as the closer proxy for human-intended dependency structure, and that subgraph is our object of study throughout. It also settles our instrument. Source-visible extraction (Section 3.1) is the correct tool for that object rather than a compromise against a fuller one. What `explicit` does and does not mean is a separate question, and Section 3.2 takes it up.

The two corpora, and the difference between them. Our human reference is mathlib itself: the 70,086 declarations of the pinned checkout that carry a located source block. Our machine corpus is a released set of 29,750 Lean proofs produced by a prover we did not run [Lin et al., 2025], and its statements come from Lean Workbook (competition problems). The larger difference is what the two are for. A mathlib declaration is general-purpose infrastructure written to be reused; a competition proof is a self-contained puzzle. That difference is a property of the released corpora rather than of provers, and it is the confound this comparison cannot remove. We state it here rather than deferring it: Section 5.2 narrows it by matching library area and Section 7, L4, bounds what the result can mean in its presence.

Centrality measures technical utility, not mathematical depth. Their third finding constrains every structural claim we make. The most heavily cited declarations in this graph are language infrastructure, and they identify a two-layer hub structure separating it from mathematical results proper. We adopt both consequences: no aggregate centrality figure without the layers separated, and "more central" never read as "less deep", an inference Li et al. [2026] contradict in print and Section 6 records us failing to respect.

Li et al. pose structural drift, as machine-generated proofs enter the library, as future work. This paper measures a machine population against their human baseline.

3 The extraction instrument, characterized

We characterize the instrument before using it. Section 3.1 states what the extractor does; the four sections after it establish what a pipeline of this kind can measure and how reliably: what

`is_explicit` actually means (Section 3.2), what retention can reach against it (Section 3.3), what displacement the extractor induces in the comparison (Section 3.4), and which axis proof length must be measured on (Section 3.5). The first two are properties of the released artifact rather than of our code, and a pipeline can consume either without receiving any signal that it has gone wrong.

3.1 Source-visible extraction at declaration scope

The extractor takes two inputs: the mathlib checkout at the pinned revision, and the set of declaration names in the published graph. It compiles nothing and elaborates nothing, which is what makes it an instrument for the explicit subgraph rather than an approximation of the full one.

Its scope is the whole declaration, statement and proof. Explicit edges come from the elaborated term of the entire declaration, so a hypothesis contributes premises. In `theorem foo (h : Nat.Prime p)`, `Nat.Prime` is an explicit target. Extracting only what follows `:=` would drop such targets while leaving them in the denominator, which is an under-scoped instrument rather than a conservative one. The release ships no source positions, so the block is located textually; the locate rate is measured rather than assumed, at 187 of 198 sampled declarations.

Within that block the identifier tokens are resolved against the declaration set, under the block-location, tokenization, resolution and prefix rules Appendix A gives in full.

Two failure modes matter for reading the results. The first is deliberate: the extractor never searches the declaration set by suffix, because resolving `mp` that way would credit every declaration in mathlib whose name ends in `.mp`. Tokens that resolve to nothing therefore stay out of the premise set entirely, and the cost of that refusal is the largest component of the shortfall below the declaration reference point of Section 3.3 (89.5%, Appendix B). The second is imposed: dot notation names a declaration only once the type of the local it projects from is known, which requires elaboration, so that residue is reported rather than resolved.

3.2 `is_explicit` does not mean "typed in source"

The dependency graph we build on [Li et al., 2026] distinguishes *explicit* edges from the much larger set of compiler-synthesized ones, and identifies the explicit subgraph as the closer proxy for human-intended dependency structure. That subgraph is our object of study, and the distinction is the right one to draw. The flag's name, however, invites a reading it does not support, and anyone building a source-level pipeline on this release should know which reading is correct before choosing a denominator.

Where the flag comes from. We consume the published graph rather than re-running its extraction; what follows is provenance, read off the release's own parser and the documented behaviour of the tool upstream of it. `is_explicit` originates as the `*` marker emitted by `lean-training-data`'s `premises` command [Morrison], and the release's parser consumes it with that meaning. The marker records that a constant occupies an *explicit argument position of the elaborated term* (a position not filled by instance resolution or unification). It is a property of the term the kernel accepts, and by construction not a claim about the surface syntax a human wrote, since it is assigned after elaboration, when the source text is gone.

The two readings come apart at the top of the graph. Elaboration inserts constants of its own. In the published graph, proof-irrelevance and rewriting plumbing (`propext`, `Eq.mpr`, `of_eq_true`) carry explicit in-edges throughout the explicit subgraph and are among its most heavily cited nodes by in-degree. No mathlib author writes any of them. They are artifacts of how `rw`, `simp` and the equation compiler construct terms, and they are explicit in the sense the flag means, sitting in explicit argument positions of terms that no human typed in that form. The same mechanism operates less visibly throughout the library, in notation-backed definitions that are never written as identifiers, in constants emitted by arithmetic normalization, and in simp-set lemmas that fire without being named.

The consequence for any source-level instrument. We take the narrower of two available claims, and it is enough. Some explicit dependencies are never written as identifiers at all (the plumbing above is the clearest case), so no instrument that reads source can recover them, and *no* source-level instrument, including a hypothetically perfect one with exact name resolution, complete namespace

and `open` handling and no tokenization loss, should be expected to reach unit retention against the explicit set. We do not claim the stronger set relation. The two sets are not nested in either direction: a name written in source need not survive as the same dependency in the elaborated term, and source text carries names that are not cited constants at all. This is why we report retention against a scope decomposition rather than against a threshold. Section 3.3 gives that decomposition as three quantities cut with one matcher at three scopes, and says what each does and does not license.

Two consequences follow for anyone building on this release. High retention rates reported for source-visible extraction elsewhere [Ji, 2026] are not comparable to ours, because a retention figure inherits the semantics of the artifact it is scored against and belongs to an (instrument, ground-truth) pair rather than to the instrument alone. And membership in the explicit set is not evidence that a human wrote the name, so it should not be used to build a retrieval target, to score a premise-selection model against "the premises a mathematician used," or to set a parser's accuracy bar. A check that costs very little catches this, and it is the same one that catches the attribute of Section 6: take the highest-in-degree nodes and ask whether a mathlib author would type them.

3.3 Retention, and what its denominator is a property of

We report retention against a decomposition rather than against a threshold, and the reason is instructive about how instruments in this setting should be scored at all.

Our original design gated the project on retention: the extractor had to clear a fixed fraction of the explicit targets in its sample or the approach was abandoned. That gate was retired before it ever returned a verdict we acted on, because its threshold turned out to encode two things that have nothing to do with our parser. The first is the elaborator behaviour of Section 3.2: part of the explicit set was never in the source text, so no fraction of it is reachable by reading source. The second is editorial: mathlib authors group related results in files, so whether a premise name happens to appear elsewhere in a declaration's file is a fact about how the library is organised. A threshold that moves when either of those changes is not measuring the instrument, and a pass it awards is not evidence about the instrument either. What replaces it is a decomposition into quantities that are each measured the same way, so that the steps between them can be read off rather than argued for.

Table 1: Retention of mathematical-layer explicit targets, same sample and same loose matcher, at three scopes. These are matcher-based reference points, not attributable bounds: a loose name match is evidence that a name occurs within the scope, not that the premise was used there, and it can equally miss a premise written under another spelling or notation.

Quantity	Denominator	Value
Whole-file reference	explicit targets, name matched anywhere in the file	0.7915
Declaration reference	explicit targets, name matched in the declaration	0.6948
Achieved retention	explicit targets, our extractor	0.6124
Efficiency	achieved against the declaration reference	0.8815

Read downward, Table 1 separates three steps of very different kinds. Targets missed even by a whole-file match are not recoverable by reading source under any spelling the matcher knows; elaborator-inserted constants are the mechanism we can name for that, and we do not claim it accounts for all of them. The step from the whole-file to the declaration reference is an organisational effect: those targets are named somewhere in the file but nowhere in the declaration that depends on them, so a per-declaration instrument cannot see them however good its name resolution is. Only the last line is a claim about our parser, and it is the figure we ask to be judged on.

The 248 targets the declaration reference point finds and the extractor misses are identified rather than merely counted, and Appendix B decomposes them: the dominant class is the definitional gap between resolving an identifier and matching a substring, and closing it entirely would mean buying retention at the cost of precision. Two properties make this a characterization rather than a disclosure. Each residual class names a mechanism. And efficiency is close to flat across tactic classes, from 0.8743 to 0.8963, so the instrument does not fail preferentially on the proof styles that distinguish the two corpora. That is a statement about mathlib proofs only, as Section 7, L1, records.

3.4 The instrument reference line

An extractor that recovers part of the explicit set displaces any comparison built on its output, so the displacement has to be quantified. We do this with a reference line, the human–machine gap the instrument produces on its own.

Two artifacts contribute and they are opposite in kind. Recall failure drops premises a declaration really cites; precision failure admits premises it does not. They do not cancel, because the corpora differ in the proof styles each artifact is worst on, so both displace the comparison the same way and the line is their sum. We therefore report it as separate terms rather than as one averaged scalar.

The recall term is standardized jointly on tactic class and length bin, on the step axis of Section 3.5. It survives that control and strengthens under it, from 0.0748 class-only to 0.0986 joint, with a bootstrap interval of (0.0678, 0.1256) and a band of (0.0762, 0.1257) across the precedence orderings of the class partition. The correction runs against our own interest, since a larger instrument artifact raises the bar our headline result has to clear.

The precision term is the weak point. By a criterion registered before the run it is uninformative rather than clean. We therefore carry it as an open item rather than folding it into the line, and Appendix C gives the diagnostics behind that decision.

Two limits on what this line licenses. Per-class efficiency is close to flat (Section 3.3), which is a stronger neutrality statement than the raw per-class miss rates suggest, but it is measured on human proofs, and nothing here establishes comparable recovery on machine proofs, which Section 7, L1, states as the paper's load-bearing assumption. And the line matters unevenly: the coverage result of Section 5.1 sits far outside the regime these artifacts operate in, while the secondary characterizations sit inside it.

3.5 Length must be measured on a wrap-invariant axis

Controlling for proof length is mandatory here. Short proofs mechanically cite fewer and more common premises, so an uncontrolled length difference can manufacture every effect we report. The choice of length *axis* is itself a measurement question with a checkable answer, and that is the reusable part of this section.

Two axes are available. On characters the corpora barely overlap, at an overlap coefficient of 0.2151. On tactic steps they nearly coincide, at 0.8563.[1] The disagreement is explained by how each corpus reaches the page. Mathlib source is hand-wrapped near a column limit and carries docstrings; a released machine proof is generated in one pass, wrapped differently or not at all, and here carries a large volume of natural-language commentary. Character counts absorb all of that; tactic counts do not, because wrapping and commentary do not change how many tactics run.

Standardizing on characters is therefore substantially standardizing on source formatting, and that is not a neutral error. On the character axis one of our secondary metrics changed sign and the recall term of Section 3.4 collapsed to a null, and both reverse on the step axis. Every length control in this paper accordingly runs on tactic steps, and character length appears once, here, as the diagnostic that exposed the problem. It is not an alternative specification, because a specification that measures wrapping is not a second opinion about proof size. Counting steps needs its own instrument and the obvious one under-records, which Appendix D measures and Section 7, L2, carries as a limitation.

The check that settles the axis is cheap. Compute the overlap coefficient on each candidate axis before choosing one. Where the axes disagree as sharply as they do here, the choice is load-bearing, and making it after seeing the comparison is a researcher degree of freedom.

4 Structural coverage: formal definitions

Let G be the explicit subgraph of the mathlib dependency graph, as characterized in Section 3.2: its nodes are declarations and its edges are the source-attributable dependencies between them. A proof p selects a set of premises $P(p) \subseteq V(G)$, the declarations it cites, and those premises span an

[1] The two rest on different bases: 0.2151 is computed on the standardization sample the joint control was cut on, 0.8563 over the full census of both corpora. The step-axis figure on the sample basis is 0.8599, so the basis is not what separates the axes.

induced subgraph $G[P(p)]$. For a corpus C we write $V(C) = \bigcup_{p \in C} P(p)$ for its *vocabulary*: the node set of the subgraph the corpus's whole proof population induces.

Coverage. Given a human corpus H and a machine corpus M, coverage is the family of quantities over $V(H)$ and $V(M)$: the two vocabulary sizes and their ratio; the Jaccard index $J = |V(H) \cap V(M)| \,/\, |V(H) \cup V(M)|$; and the human-only fraction $|V(H) \setminus V(M)| \,/\, |V(H)|$. These quantities read no edges. Computing J consults only membership, so coverage is the node-set family and breadth, below, the edge-derived one. Appendix J states what the graph does and does not supply to the coverage arithmetic, why we chose a measure that needs no aggregation, and how each quantity is reported.

Breadth. Structural breadth is the topological complement: the number of distinct communities of a fixed partition of G that a premise set touches, optionally per unit of proof length. This does read edges, since the partition is computed from them.

Which definitions need the hub-layer split. Coverage does not. It aggregates nothing, so Finding 3 does not bind, and the layer composition of each vocabulary is reported in Section G as a result in its own right rather than as a correction. Breadth and every centrality-flavoured quantity do need it. Breadth carries a second caveat, the partition skew of Appendix E, and it is reported as secondary throughout, with its three design-dependent magnitudes quoted together, never one alone.

5 Results

This section applies the definitions of Section 4 to the two corpora of Section 2: coverage and the two controls that answer the objections to it (Sections 5.1 and 5.2), then breadth (Section 5.3), reported as unstable. The composition and concentration of the machine vocabulary are reported in Appendices G and H. Two conventions hold throughout. Every figure is on the corrected premise sets. Two artifacts that resolved on the human side only (a declaration's own name and the `lemma` keyword) are removed before any comparison, at a large cost to the human count, and Section 7, L2, gives the accounting.

5.1 Vocabulary coverage

Over the full corpora the two vocabularies intersect in 1,014 premises, giving a Jaccard index of 0.0129. 98.7% of the premises mathlib's own proofs cite are never cited by any machine proof in the corpus. The asymmetry is close to containment. Only 113 premises are machine-only, so the machine vocabulary is very nearly a subset of the human one rather than a different region of the library. Two properties of that small vocabulary are reported in the appendix. It is weighted toward language infrastructure rather than mathematics, at 25.6% against the human corpus's 4.6%, leaving a mathematical working vocabulary of 838 declarations (Appendix G). And it is used very unevenly, with proofs closing in at most two steps under an arithmetic tactic set making up 23.1% of the machine corpus against 0.02% of mathlib (Appendix H). Every uncorrected instrument asymmetry we have identified narrows the gap reported here rather than widening it, so these figures are floors; Appendix F, L2, enumerates them.

The designs differ in what they are allowed to explain away. The full corpora control for nothing. Rarefaction removes the corpus-size advantage by drawing the human corpus down to the machine corpus's proof count. Common support discards the cells of the (tactic class $\times$ tactic step) grid that only one corpus occupies, so no rate is imputed where the other side has no population. The area-restricted design keeps only the regions of the library the machine vocabulary occupies at all, removing the objection that the two corpora are simply doing different mathematics. The matched design is the strictest: equal proof counts, equal tactic class, equal step bin.

The point of the table is the invariance rather than any single row. The ratio never leaves the range 18.8$\times$ to 69.8$\times$, and the human-only fraction never falls below 96.3%, under controls that variously remove corpus size, proof length, tactic style and subject area. The magnitude is design-dependent and we quote the range rather than the largest figure; the direction and order of magnitude are not. Coverage is also the most stable quantity we measure. Moving from the full corpora to common support changes the Jaccard index only in its fourth decimal place, 0.0129 to 0.0128, while that restriction discards roughly a quarter of the machine corpus. The result is therefore not an artifact

design	human	machine	ratio	Jaccard	$\lvert H \cap M \rvert$	human-only
full corpora	78,642	1,127	69.8×	0.0129	1,014	98.7%
area-restricted	**32,091**	**1,127**	**28.5×**	**0.0315**	**1,014**	**96.8%**
1:1 class- and step-matched	21,162	1,123	18.8×	0.0360	774	96.3%

Table 2: Premise vocabulary under three designs of increasing severity. Each row carries its intersection count, so its Jaccard index is recomputable from the row alone as $|H \cap M| / (|H| + |M| - |H \cap M|)$. The area-restricted row is the primary control (Section 5.2); the matched row is the strictest design we ran (Section 5.2). Two further designs are reported in the text rather than here: rarefying the human corpus to the machine corpus's size gives 44.0× (Section 5.2), and restricting to common support gives 70.0× at a Jaccard of 0.0128, against 0.0129 on the full corpora.

of which proofs happen to be comparable. Every proof on both sides of this comparison is verified. Correctness is satisfied throughout, and none of the structure in this table is visible to `pass@k`.

5.2 Corpus size and region selection

Two objections do not require argument, because both quantities can be varied directly. The first is arithmetic: more proofs find more distinct premises whatever the populations are doing, so a vocabulary compared at unequal n confounds breadth with sample size. We rarefy the human corpus (seeded draws of n proofs, distinct premises counted, mean reported), which gives its vocabulary at every corpus size. At the machine corpus's own size, 29,750 proofs, the human vocabulary is 49,601 premises, so equalizing size moves the ratio from 69.8× to 44.0× rather than to one. Further down the curve the point is sharper still. At a twentieth of the human corpus, 3,504 proofs and far fewer than the machine corpus has, the human vocabulary is 11,493 premises, an order of magnitude above what the machine corpus reaches at full size. Neither curve has saturated, so both reported vocabularies are lower bounds on their populations', and the machine curve is the nearer of the two to flattening; we do not extrapolate either beyond the corpora we have.

The second objection is the genre difference named in Section 2: if competition problems occupy a small corner of the library, a narrow machine vocabulary is a fact about which problems were posed. The control we would prefer, a same-problem comparison, does not exist. The only human Lean 4 proofs of miniF2F statements are a handful released as a prover's own few-shot prompt pool, the LeanDojo Lean 4 port carries `sorry` throughout, and the original corpus is Lean 3. We therefore restrict by region instead, keeping every top-level region that contains at least one machine premise. The rule was fixed before the run, is support-based so there is no threshold or top-k to tune, and introduces no metric. The machine vocabulary occupies 42 of the 1,595 regions the human vocabulary spans, and those regions hold 40.8% of human premises, so the restriction discards most of the human vocabulary rather than trimming it. Inside them the ratio is 28.5× at a Jaccard of 0.0315, with 96.8% of the restricted human vocabulary still untouched.

Machine leakage is zero by construction under this rule, which is what makes it primary and its registered sensitivities the check on it. The top-twelve-namespace variant gives 19.2× but discards 31.0% of the machine vocabulary on the way, and a restriction that drops nearly a third of the population it controls for is answering a different question. We report the range 19.2× to 28.5× rather than either endpoint. Neither control reaches genre, or selection *within* a region. `Nat` holds competition arithmetic and analytic number theory alike. The closest genre check we have is exploratory, and we state it here rather than only in limitations because it is the design that most constrains what the headline can mean. Mathlib's own IMO archive, against the machine corpus rarefied to the same proof count, gives 5.7×, the smallest gap of any design we ran. It was not pre-registered, it is a lower bound, it is not extraction-audited, and it is not regenerable from the released artifact (Section 7, L6). These designs bound how much of the gap corpus size and region selection explain; Section 7, L4, bounds what the result can mean in the presence of what they do not.

5.3 Structural breadth

Breadth, as defined in Section 4, counts the distinct communities of the partition a proof's premises touch, per tactic step. Machine proofs touch fewer communities per step than human ones at every

scope we can measure, and the direction is the only part of that we report: the gap is -0.607 on the full corpora, -0.780 under common support and -0.201 matched 1:1, the last two computed on 0.7659 of the machine corpus. The magnitude therefore moves by a factor of four across designs that move the coverage result in its fourth decimal place, and an earlier sign flip was an artifact of a region cut on character length, which Section 3.5 rejects as a formatting control. We report the metric as unstable, having defined it before we looked, and we do not offer it as evidence for or against the coverage result, which stands on its own controls. Appendix E carries the masses, the excluded region and the partition skew that keep it secondary.

6 A withdrawn finding: `dag_layer` and the catching check

The graph release publishes a per-declaration attribute, `dag_layer`, which we took to be distance from primitives. We built a depth metric on it, measured a machine–human gap, and promoted that gap to a stated counter-finding. The column is not a depth measure as we used it: layer zero holds 119,633 declarations whose maximum in-degree is zero (uncited leaves, not axioms), the highest layer holds `Eq.refl`, `Set` and `LE.le`, and the column correlates $+0.677$ with log in-degree, so a high value means heavily cited infrastructure and the scale runs opposite to the direction we read into it. The error was ours, and Appendix I gives the reading of the column under which it behaves as documented, together with a second hazard for other consumers of the release. What we measured is therefore a centrality proxy, and a difference in it is a statement about citation weight rather than derivation length. That is the inference Li et al. [2026] rule out in their Finding 3, which we cite in Section 2 and then walked into on a different column of the same release. We withdraw the metric rather than re-pointing it at a claim about centrality that it would support, because choosing a quantity's meaning after seeing its comparison is a researcher degree of freedom. The check that would have caught it is cheap enough to run on any released structural attribute before building on it: correlate the attribute against in-degree, then inspect its extremes and ask whether the declarations at each end match the meaning its name implies.

7 Limitations

Three objections are load-bearing, and we state them first: the corpora target different mathematics (L4), the instrument correction is measured on one side and transferred to the other (L1), and everything rests on one machine corpus (L6).

L1. The correction transfers a human rate to a larger population. Machine-side miss rates are imputed rather than measured, from per-class rates measured on human proofs and standardized onto the machine tactic mix. Arithmetic-tactic proofs are 76.4% of the machine corpus and 0.7% of mathlib, so most of the machine corpus is standardized against a stratum that is a fraction of a percent of mathlib. That rate is near-censused and precise; the exposure is that the two populations' `nlinarith` may not be the same object. In mathlib it is typically one step closing a residual goal; in the machine corpus it is frequently the whole proof. If the extractor's blind spot scales with how much of a proof the automation closes rather than with its presence, the transfer under-corrects, leaving machine proofs looking narrower than they are, which biases toward our own hypothesis. Flat efficiency across human tactic classes (Section 3.4) constrains the instrument's behaviour on mathlib proofs only; nothing measured here establishes comparable recovery on machine proofs, whose dominant stratum has almost no human counterpart. We therefore treat the correction as a lower bound and require the gap to clear a multiple of it, not merely exceed it. Elaboration-level extraction on the machine corpus would settle it. We did not do it because it means reproducing the publishers' extraction pipeline on a second corpus (building the premise tool against the same toolchain and wrapping each standalone proof as a compilable module), and because those proofs were verified against their releasers' mathlib rather than our pinned commit, so a fraction would fail on renames and deprecations, giving version skew rather than a miss rate.

Instrument limitations. Four further entries are stated in Appendix F: L2, the asymmetries left in the accounting and why every one of them narrows the reported gap; L3, how much of the explicit set source-visible extraction reaches; L3d, that Table 1's quantities are matcher-based; and L3e, that our reading of the flag is not validated against elaborated terms.

L4. The corpora target different mathematics. Machine corpora target competition statements and mathlib targets library development, so part of L1's tactic-mix difference belongs to the problem sets, not the provers. Section 5.2 narrows this by matching library *area* without removing it, because area is not *genre*. A mathlib theorem is general-purpose infrastructure; a competition problem is a self-contained puzzle. As an exploratory, not pre-registered check, mathlib's own IMO archive gives $5.7\times$, the smallest gap of any design we ran. It is a lower bound (most of those declarations cite local helper lemmas outside the graph's build target), it is not extraction-audited, and it is not regenerable from the released artifact (L6). We report it because we measured it, and publishing only the designs that give larger gaps would be the objectionable choice. We therefore claim a difference between these two released corpora under the restrictions named, not a general property of how provers use mathlib when proving comparable theorems.

L6. Scope, and what is not regenerable. One machine corpus, 29,750 proofs. These are comparisons of two distributions and not causal claims. We write "machine proofs concentrate on", never "provers are biased toward". Two families of figures carried from the prior version are *not* reproducible from the released artifact: the per-premise occurrence shares of Section H and the genre-check figures in L4. Both were computed ad hoc, neither table was committed, and no released script regenerates them. Every other number comes from a committed result file, bar Section 6's graph-audit figures, which any reader recomputes from the released graph in one pass. We flag them rather than recompute them, because recomputing after the fact is how a frozen scope stops being one.

8 Related work

Li et al. [2026] give the multilayer analysis of mathlib4 this work consumes (the graph, the explicit/synthesized distinction, the two-layer hub structure) and pose structural drift from machine-generated proofs as future work; we extend their human baseline to a machine population rather than building a baseline.

Premise retrieval asks which declarations should close a given goal and scores a ranking against the premises a human cited. Yang et al. [2023] supply that setting and its annotations, and Ji [2026] establish source-visible extraction for it, the precedent for our instrument. We run the relation backwards, asking what the selection of already-verified proofs looks like in aggregate against a library, with no right answer per goal. A retriever can score well on every goal it is given while the population it serves still occupies a small region of the library.

Three lines measure past correctness and each differs in its unit. Pham et al. [2026] score partial progress *within* a theorem; Petrov et al. [2026] characterize a proof text through proxies and pairwise comparison, needing a preference model where ours needs none; Xin et al. [2026] correlate success with transitive dependency closure size, a magnitude where coverage is a location, in a domain where the graph is a build artifact. Our corpora are the released Lin et al. [2025] proof set and mathlib itself; Zheng et al. [2022] appears for what it lacks, the human Lean 4 proofs that would permit a same-problem comparison.

9 Reproducibility and disclosure

The anonymized supplement contains the extraction and analysis source, the committed result files behind every table, the fetch scripts that obtain the input data we cannot redistribute, and the test suite in which the extraction audit asserts retention against its recorded value within a stated tolerance, so a change to the extractor fails the suite instead of silently moving a number here. Those scripts pull the published graph and the machine corpus from their public releases at pinned revisions and pin the mathlib checkout to the commit the graph was built from, every analysis runs from those pins and one fixed seed, and reported figures fall into three provenance classes that Section 7, L6, names and distinguishes. LLM assistance was used across the categories the call asks about: implementing the extraction pipeline, the metric definitions and the analysis scripts; data processing; running experiments; and drafting this paper; no number reported here is quoted from a model, and the author takes responsibility for the content, including the withdrawn finding of Section 6.

Acknowledgments and Disclosure of Funding

This work received no external funding.

References

Junye Ji. CSLibPremiseBench: Structure-Guided Premise Retrieval and Label Robustness for Lean 4 Computer-Science Theorems. *arXiv:2605.14549*, 2026.

Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The Network Structure of Mathlib. *arXiv:2604.24797*, 2026.

Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving, 2025. URL `https://arxiv.org/abs/2502.07640`.

Kim Morrison. lean-training-data. `https://github.com/kim-em/lean-training-data`. GitHub repository.

Ivo Petrov, Jasper Dekoninck, Dimitar I. Dimitrov, and Martin Vechev. Not All Proofs Are Equal: Evaluating LLM Proof Quality Beyond Correctness. *arXiv:2605.10379*, 2026. ICML 2026 AI for Math workshop.

QuocViet Pham, Elvir Karimov, Andrey Galichin, and Ivan Oseledets. TheoremBench: Evaluating LLMs on Theorem Proving in Formal Mathematics. *arXiv:2606.09450*, 2026.

Yutong Xin, Qiaochu Chen, Greg Durrett, and Işil Dillig. VeriSoftBench: Repository-Scale Formal Verification Benchmarks for Lean. *arXiv:2602.18307*, 2026.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem Proving with Retrieval-Augmented Language Models. In *Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.15626.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics. In *ICLR*, 2022.

A Extractor mechanics

Locating a declaration block. The release ships no source positions, so a declaration's block is found textually: the declaration keyword under any attribute or modifier prefix, extending to the next line that begins a top-level command. The terminator set is deliberately broad, because the two failure modes are not symmetric. Over-matching truncates a block and loses tokens at its tail, while under-matching swallows the following declaration and invents them. Failures to locate are dominated by auto-named instances, which have no textual declaration site.

Tokenization. Comments and string literals are blanked to spaces, preserving token boundaries; Lean's block comments nest, so they are scanned rather than matched by regular expression. Identifier tokens are then taken in Lean's own shape, dot-composed, with a leading dot excluded. The `.mp` in `h.mp` is projection on a local, not a reference to a root-level `mp`. A short keyword list drops tactic names before resolution. It is a precision aid rather than the safety net. Every surviving token must still resolve against the declaration set, and the list is deliberately short, since an over-long one silently excludes real premises that share a name with a tactic. Section 7, L3b, reports the one inconsistency we left in it rather than extending it after seeing which names flattered the result.

Resolution. The rule is three steps, first hit wins: the token is a fully-qualified declaration name; or it completes to exactly one declaration under a namespace prefix in scope; or it resolves to nothing and is kept aside as a bare tail. Bare tails never enter the premise set, for the reason Section 3.1 gives.

Namespace prefixes. The prefixes a bare token may complete under are the proper prefixes of the declaration's own name, together with the `open` clauses appearing in its file. Lean scopes `open` to sections and this does not, so the rule is over-permissive, and it can complete a token under a namespace not actually open at that point. The cost is precision and never recall, and it is carried in the reference line of Section 3.4.

B Residual below the declaration reference

The 248 mathematical targets that the declaration reference point finds and the extractor misses decompose into three classes. 89.5% is a definitional gap: the reference point substring-matches premise names while the extractor resolves identifiers, and a resolved identifier is a stricter object than a matched substring, so closing that gap entirely would mean abandoning resolution and buying retention at the cost of precision. 8.9% is the over-broad keyword list of Appendix A, which suppresses names that are simultaneously tactic names and library declarations; we report its cost rather than tuning the list after seeing which names it flatters. The remaining 1.6% occurs only inside comments or string literals, which the extractor strips by design. The classes are identified rather than merely counted, so each is a statement about a mechanism.

Per-class extraction efficiency (achieved retention against the declaration reference point, within each tactic class) ranges from 0.8743 to 0.8963, with the corpus-level figure 0.8815 inside that range. Section 7, L1, states what this does and does not license about the machine corpus.

C Instrument reference line: diagnostics

The precision term is carried as an open item in Section 3.4, and the diagnostics behind that decision are these. Length-controlled on the tactic-step axis the term falls to -0.0142, against -0.1917 on the character axis, a near-collapse that read alone would be good news about the instrument. By a criterion registered before the run, a control whose collapsed cells carry a large share of the machine mix is uninformative rather than clean: here they carry 24.1% of it, dominated by the arithmetic stratum of Section H. The character axis fails the companion criterion of near-disjoint supports, its overlap coefficient being 0.2151. One diagnostic points somewhere real. Across the two axes the imputed human rate is stationary while only the machine rate moves, which is the signature of a compositional correction rather than of two extrapolations converging, and that makes the step figure the more credible of the two without making it something to freeze on.

D The tactic-step proxy

Counting tactic steps needs an instrument and the obvious one under-records. The published tactic lists capture only main-path tactics; everything after `<;>`, inside `induction ... with` case branches, and in nested `(by simp ...)` term positions is missing. That is measured rather than assumed. Against a string matcher over the same declarations the disagreement is almost entirely one-directional, the matcher finding tactics the lists omit and rarely the reverse. Both corpora are therefore counted with a single fixed tactic vocabulary and one matcher. It is a size proxy, not a taxonomy, and a tactic missing from the vocabulary costs sensitivity symmetrically. Section 7, L2, states the consequence for every stratum and mix in the paper.

E Structural breadth by design

Table 3: Breadth per tactic step, machine minus human, at three scopes, with the share of the machine corpus each design retains. The sign does not move; the magnitude moves by a factor of four, which is why Section 5.3 quotes a direction and not a magnitude.

Design	Gap	Machine corpus retained
Full corpora	-0.607	all
Common support	-0.780	0.7659
Matched 1:1	-0.201	0.7659

Two things keep breadth secondary. The community partition is heavily skewed, with most communities tiny and a few holding much of the graph, so the unit is coarse before any comparison is made; and a quantity whose magnitude depends this strongly on the design cannot carry weight on its own. We report it nonetheless because the metric was defined before we looked, and omitting an unstable result would be selection rather than caution. Breadth also measures how far apart a population's

premises lie within the library, rather than how much of the library that population reaches, so it is not a second reading of coverage.

The excluded region under the last two designs is the set of cells with no human counterpart, overwhelmingly the arithmetic stratum of Section H. An earlier version of this comparison did move in sign. Restricting to a length-overlap region gave a positive gap against the full corpora's negative one. That region was cut on character length, which Section 3.5 shows to be a formatting control rather than a length control, and it retained 14.3% of the machine corpus. On the wrap-invariant axis the sign is stable at all three scopes, so the flip is an artifact of the rejected axis and not a finding.

F Limitations L2, L3, L3d and L3e

L2. Instrument asymmetries, and their direction. Human tactic use comes from the publishers' extracted lists and machine tactic use has no equivalent, so both sides are cut with the string matcher of Section 3.5 and Appendix D; the two instruments disagree materially and in one direction, and one false positive is kept rather than removed after the fact, because the tactic list was frozen before the machine corpus was touched. On the extraction side, two artifacts that resolved on the human side only (a declaration's own name and the `lemma` keyword) are removed before any reported figure, at a large cost to our own headline, and three further asymmetries are left in place: `symm` and `trans` resolve 5,222 times on the human side against 14, prefix completion supplies far more premises per proof to the human side because every machine proof carries the same fixed namespace header, and coverage is counted over both hub layers where the mathematical layer alone would give 89.6× rather than 69.8×. Every uncorrected asymmetry we have found narrows the reported gap, so Section 5.1's figures are floors. The corrections we applied cost us, and the ones we declined would have paid us. Separately, no aggregate centrality figure appears without the hub layers separated and we never infer shallowness from centrality; Section 6 reports where we broke that rule ourselves.

L3. Extraction cannot recover the explicit set in full. Sections 3.2 and 3.3 characterize this rather than bound it away. The whole-file reference point is 0.7915, which no per-declaration instrument beats, and it is a loose-matcher figure rather than an attributable bound. The reported vocabularies are instrument-filtered.

L3b/L3c. The asymmetries that remain all shrink the gap. Two artifacts that resolved on the human side only, a declaration's own name and the `lemma` keyword, are removed before any reported figure, at a large cost to our own headline. Three further asymmetries are left in place and all run the same way. `symm` and `trans` are tactic names that are also mathlib theorems, resolving 5,222 times on the human side against 14; we leave them rather than extend a keyword list after seeing which names flatter the result. Prefix completion supplies far more premises per proof to the human side, since every machine proof carries the same fixed namespace header while a mathlib declaration sits in its own. That is legitimate resolution rather than contamination. And coverage is counted over both hub layers, where the mathematical layer alone would give 89.6× rather than 69.8×. Every uncorrected asymmetry we have found narrows the reported gap, so Section 5.1's figures are floors. The corrections we applied cost us, and the ones we declined would have paid us.

L3d. The reference points are matcher-based. The quantities of Table 1 are cut with a loose name matcher: a match is evidence that a name occurs within the scope, not that the premise was used there, and we do not measure how often it credits an incidental occurrence.

L3e. Our reading of the flag is not validated against elaborated terms. The semantics established in Section 3.2 come from the release's own parser and the documented behaviour of the tool upstream of it, not from elaborating proofs ourselves, which we do not do anywhere in this work.

G A quarter of the machine vocabulary is not mathematics

The retention figures of Sections 3.3 and 3.4 are quoted on the mathematical layer, using a partition of the declaration set by provenance rather than by namespace prefix: Lean core constants, instances, coercions, kernel-level kinds and compiler-generated names are infrastructure, and the rest is mathematical content. Provenance is what makes the partition usable. A prefix rule misses every root-level

elaborator constant, and it over-captures genuine mathlib results whose names happen to sit under an infrastructure namespace. Applied to the premise vocabularies themselves rather than to retention targets, the same partition says something about the corpora.

The human vocabulary is 4.6% infrastructure: 3,579 declarations against 75,063 mathematical ones. The machine vocabulary is 25.6% infrastructure, 289 against 838, a factor of 5.6 more infrastructure-weighted. Both vocabularies place completely, with no declaration left unclassified on either side.

The machine vocabulary therefore differs from the human one in composition, not only in size. A quarter of it is the plumbing of Section 3.2 (proof-irrelevance and rewriting machinery that every Lean proof drags in and no author selects), so the mathematical working vocabulary of the machine corpus is 838 declarations. That is the figure to hold in mind next to mathlib's 75,063, since it is the set of mathematical results the released corpus actually reaches for, as opposed to the set it cites.

This is a membership count and nothing more. Each premise is classified once, by provenance, and the two sides are compared by how many fall in each class. There is no aggregation over a centrality score, so Li et al.'s Finding 3 does not bind here and we draw no inference about mathematical depth from it. A vocabulary weighted toward infrastructure is not thereby a shallower one.

This composition sits next to the concentration reported in Section H, where most of the machine corpus is short automation calls. A proof that discharges its goal in one such call still elaborates into a term, and that term carries the rewriting and proof-irrelevance machinery with it, so the two measurements are consistent with one picture of the corpus. We report them as two measurements rather than as one explained by the other. For evaluation the consequence is direct. A vocabulary that is 25.6% infrastructure and one that is 4.6% are different objects, and only the second is evidence of mathematical range, a distinction no correctness-based metric records.

The split also settles a basis question left open in Section 5.1. Coverage is reported over both layers. Counting it on the mathematical layer alone would give 89.6× rather than 69.8×, with a correspondingly higher human-only fraction, because the machine vocabulary is more infrastructure-weighted and so loses proportionally more under the restriction. We report the both-layer basis, which is the weaker of the two for our own hypothesis, and print the other here. Where two bases are defensible, the one to keep is the one that raises the bar the result has to clear.

H A characteristic concentration

Coverage counts each premise once, which makes it a set rather than a rate and is why Section 5.1 reports it with no per-proof standardization; it also makes coverage blind to usage. Counting the same premises once per occurrence gives a different picture. The machine corpus resolves 61,897 premise occurrences from its 1,127 distinct premises, against 437,062 from 78,642 on the human side, and it concentrates on a stratum essentially absent from mathlib: proofs closing in at most two steps with an arithmetic-flavoured tactic set are 23.1% of the machine corpus and 0.02% of mathlib, 14 of 70,086 declarations. We report that as a result rather than as sample attrition. It is the largest cell of the machine corpus, it has no human counterpart, and it is why the common-support design sets roughly a quarter of the machine corpus aside. Usage concentrates to match. `sq_nonneg` alone accounts for 27.0% of machine premise occurrences and the twenty-five most frequent for 68.1%, both descriptive and post-hoc, and both not regenerable from the released artifact (Section 7, L6). A machine proof resolves 2.08 premises on average against 6.24 for a human declaration. Coverage and concentration are independent and neither substitutes for the other. Heavy reuse of a few lemmas is ordinary in competition mathematics, and coverage alone would leave open that a shared vocabulary is used comparably.

I `dag_layer`: the reading under which it is correct

Section 6 records the column as a centrality proxy and the error in reading it as depth as ours. The attribute may be behaving as its producers intended: if edges run from a declaration to its dependencies and the computation seeds at in-degree-zero nodes, a column that puts uncited leaves at layer zero and `Eq.refl` at the top is exactly what the definition yields. Nothing here is a defect report against the release.

A second hazard is worth naming for other consumers. 13.8% of nodes carry a partial value that never finished propagating and is indistinguishable in the published data from a completed one, so a consumer who filters or bins on this column is mixing two populations without being told.

J What the coverage quantities do and do not read

These quantities read no edges. The objection is fair and we state it first. Computing J consults only membership, so coverage is a measure on node sets. The graph supplies the definition of a node (which names are declarations, and which layer each belongs to) rather than the arithmetic. The section headings retain the phrase *structural coverage*, and the reader should take it as naming the pair of measures defined here rather than as a claim about the coverage arithmetic. Coverage is the node-set family; breadth is the edge-derived one.

Two reporting conventions follow from that. We print the intersection count beside every Jaccard so each index is recomputable from its own row, and we give the human-only quantity as both a count and a fraction, because the two vocabularies differ in size by orders of magnitude and a ratio alone would hide which side is large. Coverage and breadth also answer different questions and can move independently: coverage asks how much of the library a population reaches, breadth how spread out that reach is, and a corpus may touch few premises scattered across many communities or many premises within one.

The choice is deliberate rather than a convenience. Any edge-reading statistic over these subgraphs requires aggregation: a centrality, a path length, a depth. Aggregation over this graph is what Li et al.'s Finding 3 shows to be uninterpretable unless the two hub layers are separated first, and Section 6 reports a second instance of the same trap on a different quantity. A membership count needs no aggregation, so it needs no such split to be read, and it survives controls that reverse or destroy the edge-reading quantities.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the three contributions the paper delivers: the instrument characterization of Section 3, the coverage results of Section 5, and the withdrawn finding of Section 6. Coverage is quoted as a range across designs rather than as a single figure, matching what Section 5 reports.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 7 is a dedicated limitations section. L1 states the instrument-transfer assumption as an assumption rather than a caveat and gives the direction of its residual error, L4 states the corpus-genre confound that no control available to us removes, and L6 states the single-corpus scope together with which reported figures are not reproducible from the artifact.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper proves no theorems. Section 4 gives measurement definitions rather than theoretical results, and each is defined constructively over the published dependency graph.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 9 gives the pinned graph revision, the pinned mathlib commit, the pinned machine-corpus revision and the single fixed seed, and Sections 3.1 to 3.5 and Section 4 specify the extractor and every metric. Reproducibility is not uniform and the

paper says so: most figures come from committed result files, a few are the graph release's own published figures and are obtained by re-fetching it at its pin, and a few are carried from the prior version of this work and are not reproducible from the artifact, which Section 7, L6, names.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Section 9 describes the anonymized supplement: the extraction and analysis source, the committed result files behind the tables, the fetch scripts that obtain the input data at pinned revisions, and the test suite. Input data is not redistributed but is fetched from its public releases, and the figures no script in the artifact regenerates are named in Section 7, L6.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: No model is trained or invoked here, so there are no hyperparameters or optimizer settings. The analogous choices are stated: the extractor's scoping and resolution rules in Section 3.1, the metric definitions in Section 4, the premise exclusion policy in Section 5.1, and the construction of each control (rarefaction, common support, area restriction, and 1:1 class-and-step matching) in Sections 5.1 to 5.2, with pins and seed in Section 9.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Where a quantity carries sampling variation we report it: the instrument reference line in Section 3.4 is quoted with a bootstrap interval and with a band over the precedence orderings of the class partition, and Section 5.2 reports the spread across seeded rarefaction draws. The headline vocabulary counts are censuses of both released corpora and carry no sampling error; Section 5.2 also states where a single seeded draw is adequate and where it is not.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.

- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The analysis is CPU-only and single-machine, with no model training or inference and no Lean invocation of our own (Section 9), but the paper reports no machine type, memory figure or wall-clock time for any step.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work analyses already-public, machine-checked proof corpora and a published dependency graph. It involves no human subjects and no personal data, anonymity is preserved in both the paper and the supplement, and LLM assistance is disclosed in Section 9.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The paper is a measurement study over released, formally verified proof corpora. It introduces no model, no deployment, and no data about people. Its foreseeable effect is on how proof automation is evaluated, which Section 1 states directly, and we identify no path from the work to a societal harm.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The artifact contains analysis code and derived result files computed over public Lean corpora. No model, no scraped data and no generative capability is released, so there is nothing carrying a misuse risk of the kind this item covers.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [No]

    Justification: Every asset is credited by citation and pinned: the dependency graph release in Section 2, the machine proof corpus and the human reference corpus in Section 8, and the exact revisions in Section 9. None of them is redistributed, since the artifact fetches rather than ships them. The paper does not, however, state the license under which each asset is published.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The only new asset is the analysis artifact, and Section 9 documents what it contains, how its input data is obtained at pinned revisions, how the paper builds from it, and which reported figures it does not regenerate.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing and no research with human subjects. Both corpora are pre-existing released artifacts.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper involves no research with human subjects, so no IRB approval or equivalent review was required.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: No LLM is a component of the method. Every measurement here is a deterministic computation over a published dependency graph and released proof corpora, and no model is invoked anywhere in the pipeline; the machine proof corpus is the object of study rather than part of the method, and was produced and verified by its publishers. LLM assistance in implementation, data processing, running experiments and drafting is separately disclosed in Section 9, as this venue's call requires, a broader requirement than this item's.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.